Python Programs

Structured Study Notes by Topic

A beginner-friendly collection of the supplied Python programs, organized into separate topics with clear headings and the original core functionality preserved.

Topics included

1. Vacation Destination Recommendation
2. Temperature Monitoring System
3. Maze Solver (Recursion and Backtracking)
4. Login and Signup System (File Handling and Hashing)
5. Math Quiz (Random Operations and Scoring)
6. Inventory Management System (Classes and Dictionary)
7. Shopping Cart with Bill Generation

1. Vacation Destination Recommendation

Program

```
destinations = {
    "warm": {
        "beach": ["Maldives", "Bahamas", "Hawaii"],
        "mountains": ["Costa Rica", "Bali", "Canary Islands"]
    },
    "cold": {
        "hiking": ["Switzerland", "New Zealand", "Norway"],
        "city": ["Paris", "Tokyo", "New York"]
    }
}

budgets = {
    "Maldives": 5000, "Bahamas": 4000, "Hawaii": 4500,
    "Costa Rica": 3500, "Bali": 3000, "Canary Islands": 4000,
    "Switzerland": 4500, "New Zealand": 4000, "Norway": 5000,
    "Paris": 3500, "Tokyo": 4000, "New York": 4000
}

budget = float(input("Enter your budget: $"))
weather = input("Climate (warm/cold): ").lower()
activity = input("Activity (beach/mountains/hiking/city): ").lower()

places = destinations.get(weather, {}).get(activity, [])
found = False

for p in places:
    if budgets[p] <= budget:
        if not found:
            print("Suggested destinations:")
            found = True
        print("-", p)

if not found:
    print("No suitable destination found.")
```

Key concepts: Concepts: nested dictionaries, user input, .get(), loops, budget filtering, if-else.

2. Temperature Monitoring System

Program

```
import random
import time

threshold = 25.0

while True:
    temp = random.uniform(20, 30)

    if temp > threshold:
        print(f"ALERT: Temperature is {temp:.2f}°C")
    else:
        print(f"Temperature is normal: {temp:.2f}°C")

    time.sleep(5)
```

Key concepts: Concepts: random.uniform(), while loop, threshold checking, formatted output, time.sleep(5). Stop the program with Ctrl+C.

3. Maze Solver — Recursion and Backtracking

Program

```
def solve(maze, x, y, end, path):
    if (x, y) == end:
        path.append((x, y))
        return True

    if (x < 0 or y < 0 or x >= len(maze) or
        y >= len(maze[0]) or maze[x][y] != 0):
        return False
```

```
    maze[x][y] = 2
    path.append((x, y))

    for dx, dy in [(1, 0), (0, 1), (-1, 0), (0, -1)]:
        if solve(maze, x + dx, y + dy, end, path):
            return True

    path.pop()
    return False

def print_maze(maze):
    for row in maze:
        print(*row)

maze = [
    [0, 1, 0, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 1],
    [0, 0, 0, 0, 0]
]

path = []
if solve(maze, 0, 0, (4, 4), path):
    for x, y in path:
        maze[x][y] = 3
    print("Path found:")
    print_maze(maze)
else:
    print("No path found.")
```

Key concepts: Concepts: recursion, boundary checks, visited cells, direction movement, path tracking, backtracking. Maze values: 0=open, 1=wall, 2=visited, 3=final path.

4. Login and Signup System — File Handling and Hashing

Program

```
import hashlib

def signup():
    email = input("Email: ")
    pwd = input("Password: ")
    confirm = input("Confirm password: ")

    if pwd == confirm:
        hashed = hashlib.md5(pwd.encode()).hexdigest()
        with open("credentials.txt", "w") as f:
            f.write(email + "\n" + hashed)
        print("Registration successful!")
    else:
        print("Passwords do not match!")

def login():
    email = input("Email: ")
    pwd = input("Password: ")
    hashed = hashlib.md5(pwd.encode()).hexdigest()

    try:
        with open("credentials.txt", "r") as f:
            saved_email, saved_hash = f.read().split("\n")

        if email == saved_email and hashed == saved_hash:
            print("Login successful!")
        else:
            print("Login failed!")
    except FileNotFoundError:
        print("Please register first.")

while True:
    print("\n1.Signup 2.Login 3.Exit")
    ch = input("Choice: ")

    if ch == '1':
        signup()
    elif ch == '2':
        login()
    elif ch == '3':
        break
    else:
        print("Wrong choice!")
```

Key concepts: Concepts: functions, input validation, hashlib, file write/read, try-except, menu loop. Security note: MD5 is not suitable for real password storage; use Argon2 or bcrypt in real applications.

5. Math Quiz — Random Operations and Scoring

Program

```
import random

difficulty = input("Difficulty (easy/medium/hard): ").lower()
operations = ['+', '-', '*']
score = 0

for i in range(1, 6):
    if difficulty == "easy":
        a, b = random.randint(1, 10), random.randint(1, 10)
    elif difficulty == "medium":
        a, b = random.randint(10, 50), random.randint(10, 50)
    else:
        a, b = random.randint(50, 100), random.randint(50, 100)

    op = random.choice(operations)

    if op == '+':
        answer = a + b
    elif op == '-':
        answer = a - b
    else:
        answer = a * b

    user = int(input(f"Q{i}: {a} {op} {b} = "))

    if user == answer:
        print("Correct!")
        score += 1
    else:
        print("Wrong! Answer:", answer)

print("Score:", score, "/5")
```

Key concepts: Concepts: random numbers, random.choice(), if-elif-else, loops, input conversion, score counting.

6. Inventory Management System — Classes and Dictionary

Program

```
class Product:
    def __init__(self, name, price, qty, category):
        self.name = name
        self.price = price
        self.qty = qty
        self.category = category

products = {}

while True:
    print("\nOptions:")
    print("1. Add a product")
    print("2. Update product details")
    print("3. Display inventory")
    print("4. Exit")
    ch = input("Enter your choice: ")

    if ch == '1':
        name = input("Enter product name: ")
        price = float(input("Enter product price: "))
        qty = int(input("Enter product quantity: "))
        category = input("Enter product category: ")

        if name not in products:
            products[name] = Product(name, price, qty, category)
            print("Product added.")
        else:
            print("Product already exists.")

    elif ch == '2':
        name = input("Enter product name to update: ")
```

```
        if name in products:
            p = products[name]
            p.price = float(input("Enter updated price: "))
            p.qty = int(input("Enter updated quantity: "))
            p.category = input("Enter updated category: ")
            print(name, "updated successfully.")
        else:
            print(name, "not found.")

    elif ch == '3':
        print("\nInventory:")
        for p in products.values():
            print(f"Name: {p.name}, Price: {p.price}, "
                  f"Quantity: {p.qty}, Category: {p.category}")

    elif ch == '4':
        break
    else:
        print("Invalid choice.")
```

Key concepts: Concepts: class, constructor (__init__), objects, dictionary, add/update/display operations, menu loop.

7. Shopping Cart — Menu, Quantities, and Bill Generation

Program

```
name = input("Name: ")
address = input("Address: ")
phone = input("Phone Number: ")

items = ["Orange", "Banana", "Tomato", "Grape", "Pear", "Lemon"]
prices = [1.00, 2.00, 3.00, 2.50, 10.00, 0.50]
cart = {}

while True:
    print("\nItems Menu")
    print("-------------------------")
    for i in range(6):
        print(i + 1, items[i], prices[i])

    print("7 Exit")
    ch = int(input("Your Choice: "))

    if ch == 7:
        break
    elif 1 <= ch <= 6:
        qty = int(input("Input Quantity: "))
        cart[ch - 1] = cart.get(ch - 1, 0) + qty
        print("Cart Updated...")
    else:
        print("Invalid choice.")

print("\n===== BILL =====")
total = 0

for i, qty in cart.items():
    amount = prices[i] * qty
    print(items[i], prices[i], "x", qty, "=", amount)
    total += amount

print("Total:", total)
```

Key concepts: Concepts: lists, dictionary cart, menu selection, quantity input, cart.get(), subtotal calculation, total bill.